BIBLIOTHÈQUE NUMÉRIQUE DIT

Rapport de Projet DevOps

Master 1 Intelligence Artificielle
Dakar Institute of Technology

Groupe 2
AHOGA Josias · BAH Mamoudou · BAMBA Yannick · DIOP Seynabou

Année académique 2024 – 2025

Table des matières

1 Introduction

2 Architecture du projet

3 Technologies utilisées

4 Microservices

5 Frontend Vue.js

6 Base de données

7 Fonctionnalités détaillées

8 Guide d'utilisation de la plateforme

8.1 Premier lancement

8.2 Connexion et inscription

8.3 Dashboard administrateur

8.4 Gestion des livres

8.5 Gestion des utilisateurs

8.6 Gestion des emprunts

8.7 Validation manuelle des emprunts

8.8 Mot de passe temporaire

9 Docker et Docker Compose

10 Pipeline CI/CD Jenkins

11 Conclusion

1. Introduction

Ce rapport présente le projet de Bibliothèque Numérique développé dans le cadre du cours de DevOps du Master 1 en Intelligence Artificielle au Dakar Institute of Technology (DIT). Le projet consiste en une plateforme de gestion de ressources documentaires basée sur une architecture microservices. Elle permet aux étudiants et enseignants du DIT d'emprunter des livres, et à l'administrateur de gérer le catalogue, les utilisateurs et les emprunts.

L'objectif pédagogique est de mettre en pratique les concepts DevOps : conteneurisation avec Docker, orchestration avec Docker Compose, intégration continue avec Jenkins, et développement d'APIs REST avec FastAPI.

2. Architecture du projet

Le projet suit une architecture microservices composée de cinq composants :

- Service Livres (port 8001) : CRUD catalogue, gestion des exemplaires et disponibilité
- Service Utilisateurs (port 8002) : authentification, gestion des comptes, création d'accès temporaires, désactivation de comptes
- Service Emprunts (port 8003) : prêts, retours, validation manuelle/automatique, détection des retards, limite 2 emprunts par utilisateur
- Frontend Vue.js (port 80) : interface web monofichier servie par Nginx, adaptée selon le rôle
- PostgreSQL (port 5432) : base de données unique partagée par les trois microservices

Les microservices communiquent entre eux via des appels HTTP internes (httpx).

3. Technologies utilisées

Technologie	Version	Utilisation
Python	3.12	Langage backend
FastAPI	0.115	Framework API REST
SQLAlchemy	2.0	ORM
PostgreSQL	16	Base de données
Vue.js	3 (CDN)	Framework frontend
Docker	Latest	Conteneurisation
Docker Compose	Latest	Orchestration
Jenkins	Latest	Pipeline CI/CD
Nginx	Alpine	Serveur web frontend
httpx	0.27	Communication inter-services

ReportLab	Latest	Génération PDF

4. Microservices

4.1 Service Livres

Gère le catalogue de la bibliothèque. Chaque livre possède un titre, auteur, ISBN unique, année, genre, quantité totale d'exemplaires et nombre d'exemplaires disponibles. Lors d'un emprunt le nombre disponible est décrémenté ; lors d'un retour il est incrémenté.

4.2 Service Utilisateurs

Gère l'authentification et les comptes. Trois types : étudiant (matricule format NE2510081), professeur (pièce d'identité), personnel administratif. Authentification par email et mot de passe hashé SHA-256 avec token JWT maison. Fonctionnalités clés : création de comptes avec mot de passe temporaire (changement obligatoire à la première connexion), désactivation douce des comptes (historique conservé).

4.3 Service Emprunts

Gère les prêts de livres avec deux modes de validation. En mode automatique, l'emprunt est validé et le stock décrémenté immédiatement. En mode manuel, l'emprunt est créé en attente (le stock n'est pas touché) et l'admin doit valider depuis l'interface. Durée fixe de 15 jours, maximum 2 emprunts simultanés par utilisateur, détection automatique des retards.

5. Frontend Vue.js

Interface monofichier HTML utilisant Vue.js 3 via CDN. L'interface s'adapte au rôle de l'utilisateur connecté (administrateur ou membre). Le token d'authentification est transmis automatiquement dans le header Authorization de chaque requête.

6. Base de données

Une seule instance PostgreSQL partagée. Tables créées automatiquement via SQLAlchemy :

Table	Champs principaux
livres	id, titre, auteur, isbn, annee, genre, quantite, disponible
utilisateurs	id, nom, prenom, email, mot_de_passe, type, role, actif, mot_de_passe_temporaire
emprunts	id, utilisateur_id, livre_id, date_emprunt, date_retour_prevue, date_retour_reelle, retard, valide
config	cle, valeur (ex: auto_validation = true/false)

7. Fonctionnalités détaillées

- Authentification par email et mot de passe avec token
- Inscription avec confirmation du mot de passe et validation conditionnelle (matricule étudiant, pièce d'identité enseignant)
- Changement de mot de passe obligatoire à la première connexion (mot de passe temporaire)
- Désactivation douce des comptes membres (conservation de l'historique d'emprunts)
- Blocage de la désactivation si des emprunts sont en cours
- Gestion des livres avec quantité d'exemplaires (ex: 2/3 disponibles)
- Emprunt de livres avec durée fixe de 15 jours
- Validation automatique ou manuelle des emprunts (toggle admin persisté en base)
- Maximum 2 emprunts simultanés par utilisateur (emprunts validés uniquement)
- Détection automatique des retards avec calcul du nombre de jours
- Dashboard adapté au rôle avec statistiques cliquables
- Page Emprunts avec trois onglets : En cours, En attente, Tous
- Carte de notification pour les emprunts en attente de validation
- Recherche de livres par titre, auteur ou ISBN
- Pipeline CI/CD Jenkins compatible Windows et Linux

8. Guide d'utilisation de la plateforme

Cette section présente les principales fonctionnalités de la plateforme illustrées par des captures d'écran de l'interface.

8.1 Premier lancement

Après avoir cloné le dépôt et lancé Docker Compose :

- git clone https://github.com/Sihiney/bibliotheque-dit.git
- cd bibliotheque-dit
- docker compose up --build
- Ouvrir http://localhost dans le navigateur

Au premier lancement, aucun compte n'existe. Il faut créer le compte administrateur manuellement via l'API Swagger à http://localhost:8002/docs, endpoint POST /utilisateurs, avec le corps suivant :

```
{"nom":"Admin","prenom":"DIT","email":"admin@dit.sn","mot_de_passe":"admin123","type":"pe
rsonnel_administratif","role":"admin"}
```

8.2 Connexion et inscription

La page d'accueil présente le logo DIT à gauche et le formulaire de connexion/inscription à droite. L'utilisateur se connecte avec son email et son mot de passe. Le lien « Mot de passe oublié » renvoie vers admin@dit.sn.

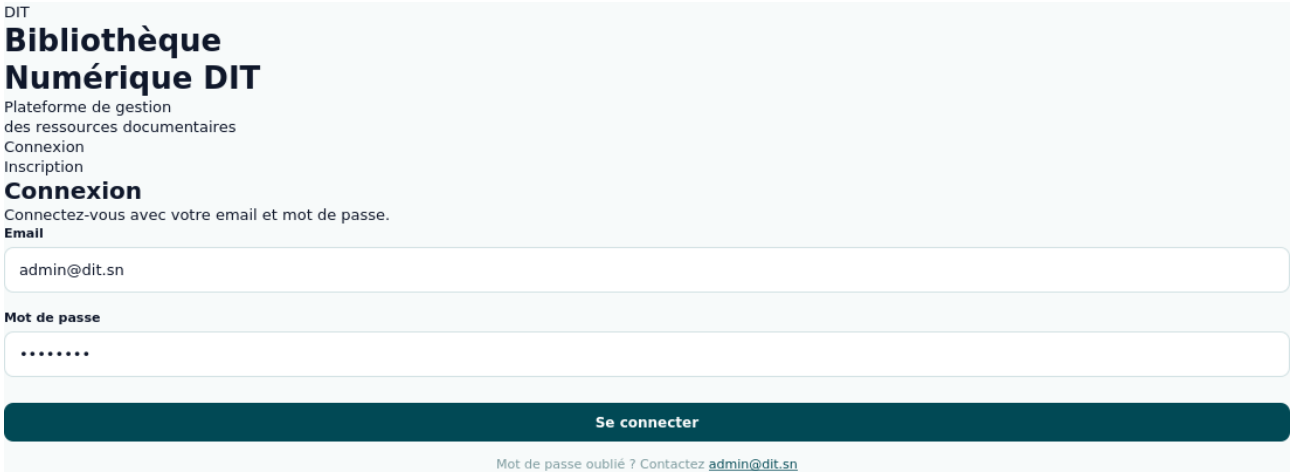

Figure 1 – Page de connexion / inscription

8.3 Dashboard administrateur

Le dashboard administrateur affiche cinq statistiques cliquables : Livres, Membres, Emprunts en cours, Retards, et En attente de validation. La carte « En attente » affiche un badge rouge d'alerte quand des demandes sont en attente. Le toggle de validation automatique est branché sur l'API et persiste entre les sessions.

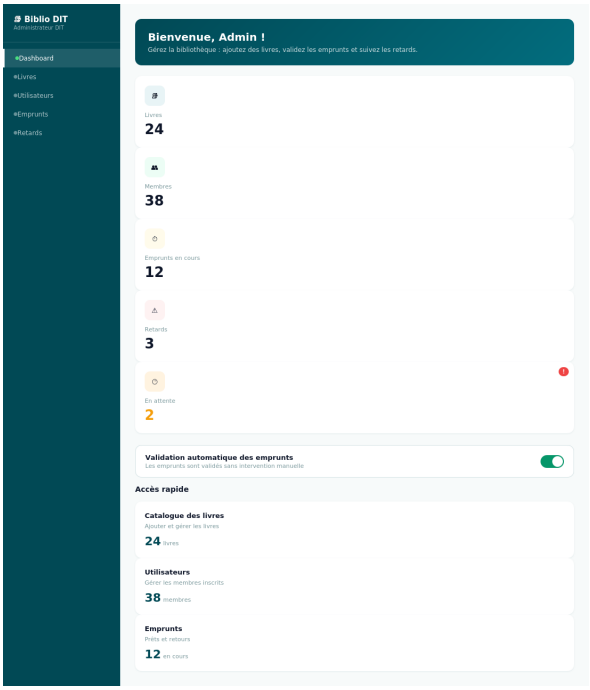

Figure 2 – Dashboard administrateur avec les 5 statistiques et le toggle de validation

Le dashboard membre affiche uniquement ses propres statistiques : livres disponibles, emprunts en cours et retards, ainsi que le tableau de ses emprunts actifs.

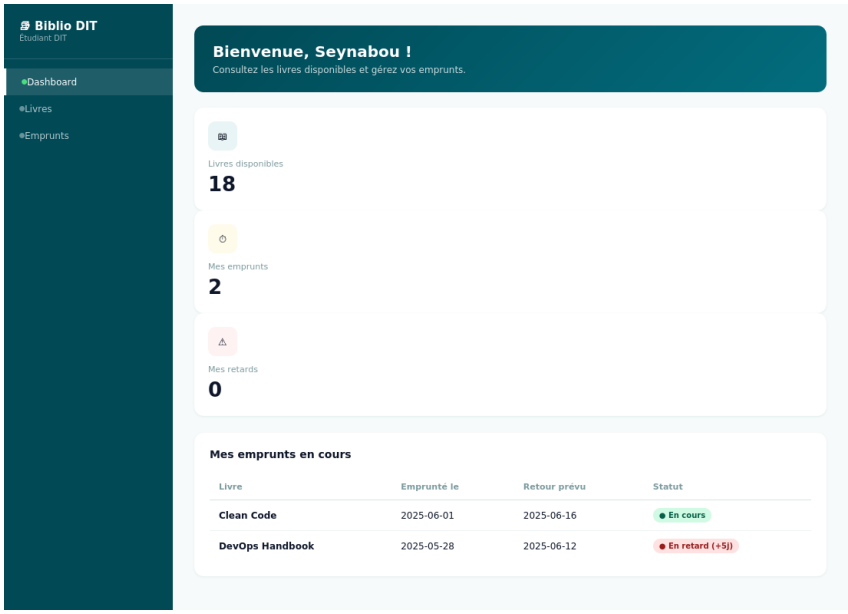

Figure 3 – Dashboard membre avec emprunts personnels

8.4 Gestion des livres

La page Livres affiche le catalogue complet avec une barre de recherche (titre, auteur, ISBN). Chaque livre indique sa disponibilité (ex: 2/3 disponibles). L'administrateur peut ajouter, modifier ou supprimer des livres. La disponibilité est mise à jour automatiquement à chaque emprunt ou retour validé.

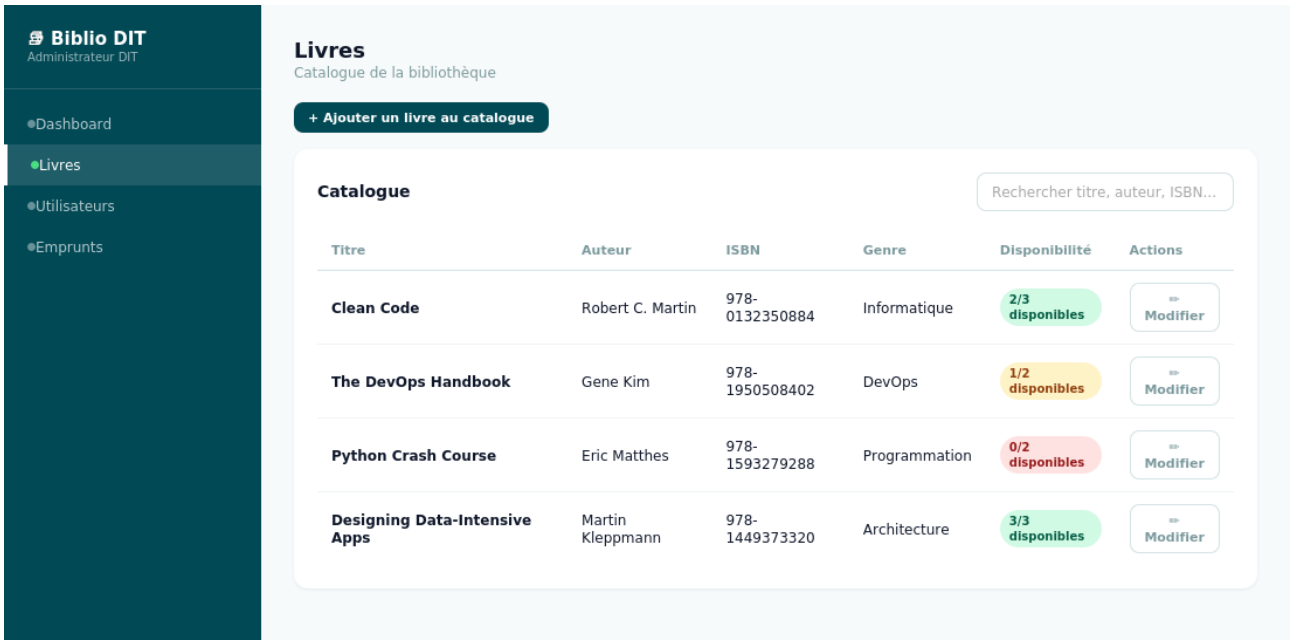

Figure 4 – Catalogue des livres avec indicateurs de disponibilité

8.5 Gestion des utilisateurs

La page Utilisateurs liste tous les membres inscrits. L'administrateur peut modifier les informations d'un membre ou désactiver son compte. La désactivation est bloquée si des emprunts sont en cours, avec un message explicite indiquant le nombre d'emprunts restants. Un compte désactivé conserve son historique complet d'emprunts.

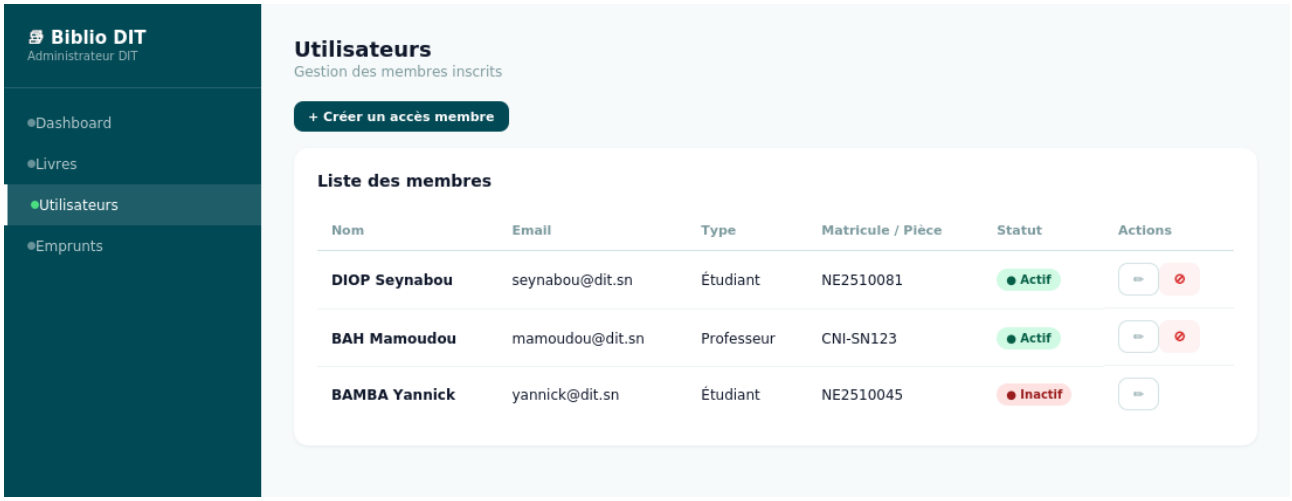

Figure 5 – Liste des membres avec statut actif/inactif

Pour créer un accès, l'admin remplit le formulaire avec les informations du membre et définit un mot de passe temporaire que l'utilisateur devra changer à sa première connexion.

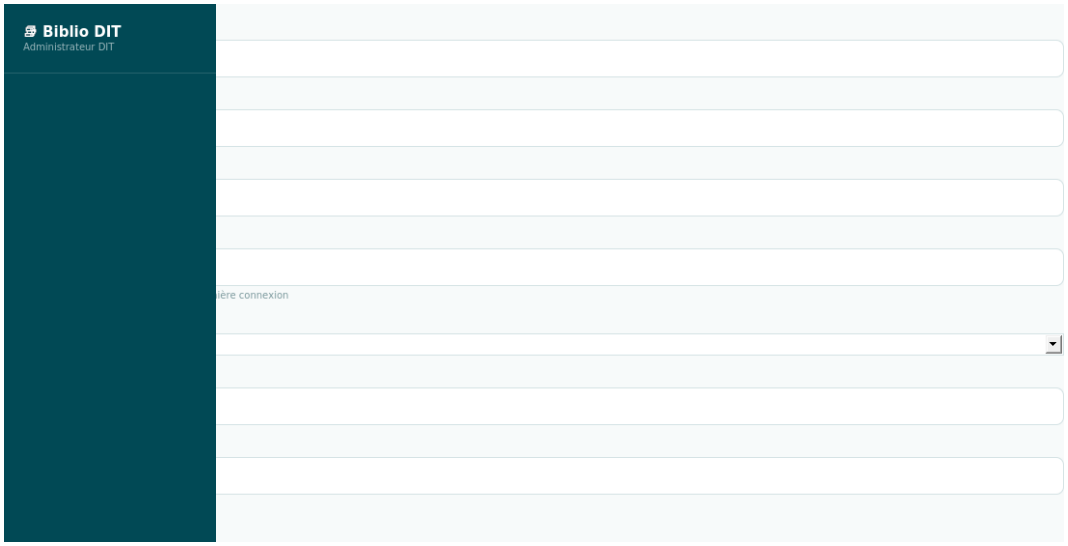

Figure 6 – Formulaire de création d'un accès membre avec mot de passe temporaire

8.6 Gestion des emprunts

La page Emprunts propose trois onglets. Chaque onglet affiche un compteur en temps réel. L'onglet actif par défaut est « En cours » (cas d'usage principal). L'onglet « En attente » affiche un badge d'alerte rouge quand des demandes attendent une validation.

Figure 7 – Page Emprunts avec les trois onglets : En cours, En attente, Tous

8.7 Validation manuelle des emprunts

Quand la validation manuelle est activée (toggle désactivé sur le dashboard), les demandes d'emprunt sont créées avec le statut « En attente ». Le stock du livre n'est pas décrémenté tant que l'admin n'a pas validé. L'onglet « En attente » affiche un bouton ✓ Valider pour chaque demande.

Figure 8 – Onglet En attente avec bouton de validation admin

Sur le dashboard, la carte « En attente » affiche un badge rouge d'alerte et est cliquable pour accéder directement à l'onglet de validation. En mode manuel, le toggle affiche un fond rouge et le message explicatif change.

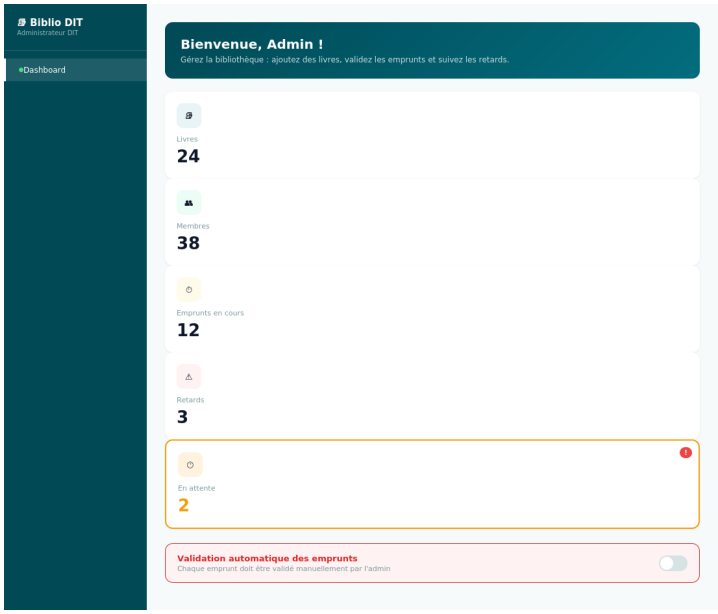

Figure 9 – Dashboard avec toggle en mode validation manuelle et alerte En attente

8.8 Changement de mot de passe obligatoire

Lorsqu'un compte est créé par l'admin avec un mot de passe temporaire, l'utilisateur est accueilli par un modal bloquant à sa première connexion. Il est impossible de fermer ce modal ou d'accéder au dashboard sans avoir changé le mot de passe. Après validation, le flag mot_de_passe_temporaire passe à False en base et l'accès est accordé.

Figure 10 – Modal bloquant de changement de mot de passe à la première connexion

9. Docker et Docker Compose

Chaque composant possède son propre Dockerfile basé sur une image slim. Le docker-compose.yml orchestre 5 conteneurs sur un réseau Docker dédié :

- postgres : image postgres:16-alpine avec healthcheck pg_isready
- service-livres : FastAPI sur port 8001, démarre après postgres
- service-utilisateurs : FastAPI sur port 8002, démarre après postgres
- service-emprunts : FastAPI sur port 8003, démarre après postgres + les deux autres services
- frontend : Nginx Alpine sur port 80, sert le fichier index.html unique

Commandes essentielles :

Lancer l'application	`docker compose up --build`
Arrêter	`docker compose down`
Voir les logs	`docker compose logs --tail=50`
Reconstruire un service	`docker compose build service-emprunts`

10. Pipeline CI/CD Jenkins

Le Jenkinsfile est compatible Windows et Linux grâce à la fonction isUnix(). En cas d'échec, le bloc post { failure } est protégé par un try/catch pour éviter les crashs liés à l'absence de shell sur Windows.

Étape	Description
1. Récupération du code	Clone du dépôt GitHub
2. Vérification structure	Contrôle des 12 fichiers essentiels
3. Build Docker	Construction des 4 images (avec diagnostic docker --version)
4. Arrêt ancienne version	Suppression des anciens conteneurs (try/catch sur Windows)
5. Déploiement	Lancement des nouveaux conteneurs
6. Vérification santé	Test des endpoints /health des 3 services + frontend
7. Tests fonctionnels	Validation APIs POST/GET + flux mot de passe temporaire

11. Conclusion

Ce projet nous a permis de mettre en pratique les concepts DevOps fondamentaux : architecture microservices, conteneurisation Docker, orchestration Docker Compose, pipeline CI/CD Jenkins compatible multi-plateformes, et APIs REST FastAPI.

La plateforme couvre les besoins d'une bibliothèque académique avec des fonctionnalités avancées : gestion du catalogue avec exemplaires, authentification sécurisée avec token, changement de mot de passe obligatoire à la première connexion, désactivation douce des comptes membres, système de validation des emprunts à double mode (automatique/manuel), interface adaptée aux rôles avec notifications en temps réel.